

Tools — MasterFetch

MasterFetch provides rich web content extraction, multi-engine search, and same-domain crawling with a content cache. Outputs carry envelope signals (`content_ok`, `page_type`, `next_action`, `source_type`, `is_official`, `content_age_days`, `is_stale`).

Tool	Description
<code>mf_fetch</code>	Rich URL/PDF extraction with envelope signals.
<code>mf_search</code>	Keyless multi-engine web search.
<code>mf_crawl</code>	Best-first same-domain crawl.
<code>mf_cache_clear</code>	Clear the content cache.
<code>mf_screenshot</code>	Capture a page as a screenshot.
<code>mf_version</code>	Return integration version info.

Use cases: extracting article content, searching multiple engines in parallel, crawling a domain.

Visibility switch: `masterfetch` (see `docs/howtos/tool-visibility.md`).

`mf_search` runs DuckDuckGo, Brave, OpenAlex, and Wikipedia in parallel (plus optional LangSearch / Tavily / Perplexity / Exa / Serper engines when their keys are configured). `exclude_engines` removes named backends before any request is dispatched; naming every configured engine returns an explicit “all engines excluded” result. An optional `engine` restricts the search to a single backend.

Set `respect_robots: true` on `mf_fetch` / `mf_crawl` when unsure whether a site allows crawling; do not hammer the same domain with repeated calls.

`mf_fetch`

Fetch any URL or PDF with automatic content extraction.

Arguments

Argument	Type	Required	Description	Typical value
<code>url</code>	string	one of <code>url</code> / <code>urls</code>	Single URL to fetch	"https://example.com/article"
<code>urls</code>	array	one of <code>url</code> / <code>urls</code>	Array of URLs for parallel bulk fetch (up to 8 concurrent)	["https://a.com", "https://b.com"]
<code>format</code>	enum	no	markdown (default), <code>html</code> , <code>text</code> , <code>raw</code>	"markdown"
<code>css_selector</code>	string	no	Narrow extraction scope	"article"
<code>focus</code>	string	no	Query string for BM25 post-extraction filtering (no re-fetch)	"memory safety"
<code>max_content_chars</code>	integer	no	Maximum content characters (min 500)	40000

Argument	Type	Required	Description	Typical value
offset	integer	no	Character offset to resume pagination	40000
include_links	boolean	no	Classify outgoing links into citations/navigation/external	false
respect_robots	boolean	no	Check robots.txt before fetching	false
cache_ttl	integer	no	Cache TTL in seconds (0 bypasses cache)	3600

Example:

```
mf_fetch url="https://example.com" format="markdown"
mf_fetch urls=["https://a.example.com","https://b.example.com"] max_content_chars=20000
```

mf_search

Keyless multi-engine web search; engines run in parallel and results are merged, deduplicated, and ranked with `relevance_score`, `fetch_relevance`, and `engines_consensus`.

Arguments

Argument	Type	Required	Description	Typical value
query	string	yes	Search query	"zero-copy parsing in rust"
engine	enum	no	Restrict to one backend: openalex, wikipedia, langsearch, tavily, perplexity, exa, serper	—
exclude_engines	array	no	Engine names to drop before dispatch	["openalex"]
site	string	no	Restrict results to a domain	"rust-lang.org"
exclude_sites	array	no	Domains to exclude	["pinterest.com"]
freshness	enum	no	day, week, month, year	"week"
max_results	integer	no	Cap after merge/dedup (1–500)	6
per_engine_results	integer	no	Cap per engine before merge (1–200)	75
page	integer	no	Result page (0–10)	0

Example:

```
mf_search query="rust async patterns" max_results=10
mf_search query="rust async patterns" exclude_engines=["openalex"]
mf_search query="site:doc.rust-lang.org ownership rules" site="doc.rust-lang.org"
```

mf_crawl

Best-first, same-domain crawl starting from a URL. Each page is returned as markdown with `content_ok` and `page_type` envelope signals.

Arguments

Argument	Type	Required	Description	Typical value
<code>url</code>	string	yes	Start URL	"https://docs.rs/serde"
<code>max_pages</code>	integer	no	Maximum pages to fetch	10
<code>max_depth</code>	integer	no	Maximum crawl depth from start URL	2
<code>max_total_chars</code>	integer	no	Total character budget across pages	200000
<code>deadline_ms</code>	integer	no	Time budget in milliseconds	120000
<code>focus</code>	string	no	Query string for scoring/filtering pages	"tutorial"
<code>sitemap</code>	boolean or "auto"	no	Use sitemap (true), use if available ("auto"), or pure BFS (false)	false
<code>discover_only</code>	boolean	no	Return discovered URL map without fetching content	false
<code>crawl_urls</code>	array	no	Second-phase selective crawl: fetch only this subset of URLs	—
<code>respect_robots</code>	boolean	no	Check robots.txt before fetching	false

Example:

```
mf_crawl url="https://doc.rustup.rs" max_pages=15 max_depth=2 focus="getting started"
```

mf_cache_clear

Clear the masterfetch content cache.

Arguments

Argument	Type	Required	Description	Typical value
all	boolean	no	true wipes all entries; false/omitted purges only expired entries	false

mf_screenshot

Capture a page as a screenshot image. The integrated Rust runtime has no headless browser engine, so this currently returns an error recommending `mf_fetch` for text-based extraction (or use the browser tool for CDP screenshots).

Arguments

Argument	Type	Required	Description	Typical value
url	string	yes	URL to capture	"https://example.com"
width / height	integer	no	Viewport size in pixels	1280 / 800
full_page	boolean	no	Capture the full scrollable page	false

mf_version

Return the masterfetch integration version, the ragent version, and a brief description of the tool set. Makes no network calls and always succeeds.

Arguments: none.